

Nearest-neighbor Gaussian processes for Bayesian hierarchical models in NIMBLE: Scalable densities, samplers and prediction in BayesNSGP

Fabian R. Ketwaroo ¹ and Daniel Turek ²

¹ Swiss Ornithological Institute, Sempach, Switzerland ² Lafayette College, Easton, PA, United States ¶
Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Neea Rusch](#) 

Reviewers:

- [@Mari1988](#)
- [@csetraynor](#)
- [@xulinpan](#)

Submitted: 29 August 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/))

Summary

Ecologists, epidemiologists and environmental scientists routinely need to account for the fact that observations taken close together in space tend to be similar. Gaussian processes (GPs) are the standard statistical tool for this, but they scale poorly for large datasets: fitting a GP to M locations requires operations that grow as M^3 , so even a few thousand sites can make a model impractical to fit. The nearest-neighbor Gaussian process (NNGP, [Datta et al., 2016](#)) removes this bottleneck by letting each location depend on only a small number, k , of nearby locations, which yields a valid probability model whose cost grows linearly in M .

This paper describes new NNGP functionality added to the BayesNSGP package ([Risser & Turek, 2020](#)) for the NIMBLE probabilistic programming system ([de Valpine et al., 2017](#)). The additions let a user build the neighbor structure for a set of coordinates, simulate a spatial surface, insert an NNGP-distributed spatial random effect directly into a hierarchical model written in the BUGS language, sample it with a purpose-built Markov chain Monte Carlo (MCMC) algorithm, and predict at unobserved locations. The spatial process is formulated as a user-defined probability distribution. It can therefore be combined with any likelihood expressible in the BUGS language: count, occupancy, capture–recapture, disease-prevalence and integrated models.

Statement of need

Hierarchical spatial models in ecology and epidemiology are usually non-Gaussian: counts, detections and survival outcomes are observed, and a latent spatial field enters through a link function. In this setting the spatial random effects cannot be integrated out analytically, so an MCMC algorithm fitting the model must update the entire field at every iteration. Two issues then need attention. First, evaluating the spatial density is $\mathcal{O}(M^3)$ using a full GP model. Second, the sampling algorithm matters as much as the GP density itself. The NIMBLE package provides conditional autoregressive distributions for areal data, but no GP approximation for point-referenced data, and its automatic sampler assignment is necessarily generic: a multivariate node receives a block Metropolis-Hastings random-walk sampler. This is a sensible general-purpose default, but is not tailored to a high dimensional spatial field, where joint proposals are very rarely accepted.

A fast density is therefore not sufficient on its own: an efficient sampler is needed alongside it, and implementations that provide both are generally tied to a specific model family. The components described here target researchers who need a scalable spatial random effect as a single component of a larger, custom hierarchical model, and who need that component to be both statistically exact and computationally efficient within the model they have written.

State of the field

Several mature implementations of NNGP and related Vecchia-type approximations exist. `spNNGP` (Finley et al., 2022) is the reference R implementation: it is highly optimised, offers OpenMP parallelisation, and handles Gaussian responses and binomial responses via a Pólya-Gamma augmented Gibbs sampler. Its scope, however, is spatial regression: users cannot embed its NNGP inside an arbitrary hierarchical model. `GpGp` (Guinness, 2018) and `BRISC` (Saha & Datta, 2018) provide fast maximum-likelihood and bootstrap inference for Vecchia approximations, but are not Bayesian. `R-INLA` (Lindgren et al., 2011; Rue et al., 2009) supports very large latent Gaussian models through a stochastic partial differential equation representation, but is restricted to the latent Gaussian model class and does not give the user control over the sampling algorithm.

`BayesNSGP` (Risser & Turek, 2020) already contains an NNGP approximation, which is available through its `nsgpModel()` interface for (non)stationary GP regression, where the approximation serves as the likelihood of the observed data: the `dmnorm_nngp()` distribution is used for the response vector, where the covariance parameters are allowed to vary in space. Prediction follows the same idiom. `nsgpPredict()` supports the NNGP likelihood, but operates on a fitted `nsgpModel()` object, reading the constants and submodel specification that the wrapper encodes. This is a coherent design for the problem it addresses. The alternative case, where the process must remain a latent field because the observation model is non-Gaussian, calls for different machinery: a density which can be evaluated on a latent node, a simulator, a sampler suited for updates, and a prediction algorithm that conditions on posterior draws. A user who takes the existing density and writes such a model by hand receives NIMBLE's default block sampler for the field. Rather than create a competing package, we contribute to `BayesNSGP` a complementary, user-facing layer for the stationary case, designed to be composed by hand inside BUGS code, together with the simulation, sampling and prediction machinery that a latent-field formulation requires. The new functions are named `dmnorm_NN_GP()`, `rmnorm_NN_GP()`, the sampler `RW_NN_GP`, and the prediction function `NNGP.pred()` so that they coexist with the package's existing functions rather than replacing them.

Software design

Our contribution comprises three groups of functions and rests on three design decisions.

A compressed representation instead of sparse matrices. The NNGP implies a sparse Cholesky decomposition of the precision matrix $\tilde{\mathbf{K}}^{-1} = (\mathbf{I} - \mathbf{A})^\top \mathbf{D}^{-1} (\mathbf{I} - \mathbf{A})$, where $\mathbf{A}$ is strictly lower triangular with at most k non-zero entries per row and $\mathbf{D}$ is diagonal. NIMBLE's domain-specific language has limited support for sparse matrix types, so rather than storing $\mathbf{A}$ and $\mathbf{D}$ sparsely, `computeAD()` packs their non-zero elements into a dense $M \times (k + 1)$ matrix: columns 1: k hold the neighbor weights, and column $k + 1$ holds the conditional variances. This keeps every operation inside the subset of the language that NIMBLE compiles to C++, gives contiguous memory access in the inner loops, and — because `computeAD()` is a `nimbleFunction` — allows the weights to be declared as a deterministic node within the model, so that they are recomputed automatically whenever the range (or variance) parameter is updated.

The process as a distribution, not a wrapper. `dmnorm_NN_GP(mu, AD, neighbors.id)` is registered as a user-defined multivariate distribution for the latent spatial field $\boldsymbol{\omega} = (\omega_1, \dots, \omega_M)$, evaluating the log-density as a sum of standardised conditional residuals via `computeQF()`, never forming a covariance matrix. The deliberate trade-off is that the user must construct and pass the neighbor structures themselves — `computeNeighbors()` returns coordinates sorted along the horizontal axis (Vecchia, 1988), a distance matrix, neighbor indices and neighbor distances. Sorting is exposed rather than hidden precisely because the user must remap their response vector to the sorted order, and automatic (under the hood) sorting introduces potential for error. Distances are computed through `sf` (Pebesma, 2018),

so projected or geographic coordinate reference systems are both supported. `rmnorm_NN_GP()` simulates the field sequentially in $\mathcal{O}(Mk)$ operations (Datta, 2022), drawing each value from its conditional distribution given its neighbors. Simulating a GP by the usual route requires a Cholesky factorisation of the full $M \times M$ covariance matrix with cost $\mathcal{O}(M^3)$, so this is a practical advantage in its own right as well as a source of sensible MCMC starting values.

A sampler that exploits the directed acyclic graph. A generic scalar random-walk sampler must reevaluate the full spatial density after each individual scalar proposal, which is $\mathcal{O}(M)$. `sampler_RW_NN_GP()` instead recognises that changing ω_i alters only its own conditional density and those of its *reverse* neighbors — the nodes that condition on it. `get_single_reverse_neighbors()` and `RWNGP_setup()` resolve this local graph once, in R, at MCMC configuration time, and translate it into explicit node addresses; the compiled sampler then performs an $\mathcal{O}(k^2)$ update, the cost of which is independent of M . Proposal scales adapt following Shaby & Wells (2011). The trade-off is that the sampler must be assigned node by node with the neighbor matrix supplied at sampler assignment.

Prediction is handled as a post-processing step by `NNGP.pred()`, which follows Algorithm 2 of Finley et al. (2019): for each new location it finds the k nearest training neighbors under the same ordering rule, and draws are generated using the resulting univariate conditional distribution.

Research impact statement

A [reproducible vignette](#) fits a spatial Poisson generalised linear model with a latent NNGP field ($M = 1000$ locations, $N = 900$ for training, $P = 100$ for validation, $k = 15$) and benchmarks the three sampling strategies. NIMBLE's default block sampler left roughly 85% of the 900 spatial random effects with $\hat{R} > 1.2$. Assigning scalar random-walk samplers resolved convergence for every node but ran about 7.5 times slower in wall-clock time, for a median gain of about 18-fold in effective sample size per second. The `RW_NN_GP` sampler also achieved convergence at every node while running only about 1.9 times slower than the block sampler, giving a median efficiency roughly 69 times that of the block sampler and 3.8 times that of the generic scalar sampler. Posterior means were closely comparable to those from the generic sampler (median relative bias -0.093 versus -0.094), confirming that the local update targets the same posterior, and out-of-sample predictive counts at the 100 held-out locations had a median relative bias of 0.031. Because the per-update cost depends on k and not M , the advantage grows with dataset size.

These components were developed to meet a concrete research need and are already in use. They provide the spatial machinery of a Bayesian spatially explicit integrated population model (Ketwaroo et al., 2026), which combines population count and capture–recapture data from many sampling locations to estimate spatio-temporal demographic rates such as survival and recruitment. There, the NNGP models residual spatial autocorrelation and generates the spatial predictions in an analysis of Gray Catbird (*Dumetella carolinensis*) data from the North American Breeding Bird Survey and the Monitoring Avian Productivity and Survivorship program across the eastern coast of North America. That application is precisely the case the design targets: a latent spatial field embedded in a large, custom, non-Gaussian hierarchical model that no existing NNGP package can express, sampled node by node across many thousands of MCMC iterations.

In accordance with JOSS policy on co-publication, we note that Ketwaroo et al. (2026) is a related methodological manuscript currently available as a preprint; the present paper describes the software, not those research results.

136 AI usage disclosure

137 Generative AI was used in preparing this manuscript. Claude (Anthropic) was used to produce
 138 an initial draft of the manuscript text from the existing package vignette and the Roxygen
 139 documentation of the contributed functions, and for revisions. All statistical and algorithmic
 140 design decisions, the implementation, the benchmarks reported above and the final text are
 141 the work of the human authors, who reviewed, edited and validated all AI-assisted output and
 142 are responsible for its accuracy.

143 Acknowledgements

144 We thank the developers of NIMBLE for an extensible design that made this contribution possible,
 145 Jaume Badia for testing the code, and Michael Schaub for reading the manuscript. This work
 146 was supported by the Swiss National Science Foundation (grant 215689). The funder had no
 147 involvement in the design, implementation or reporting of the software.

148 References

- 149 Datta, A. (2022). Nearest-neighbor sparse Cholesky matrices in spatial statistics. *Wiley*
 150 *Interdisciplinary Reviews: Computational Statistics*, 14(5), e1574. [https://doi.org/10.](https://doi.org/10.1002/wics.1574)
 151 [1002/wics.1574](https://doi.org/10.1002/wics.1574)
- 152 Datta, A., Banerjee, S., Finley, A. O., & Gelfand, A. E. (2016). Hierarchical nearest-neighbor
 153 Gaussian process models for large geostatistical datasets. *Journal of the American Statistical*
 154 *Association*, 111(514), 800–812. <https://doi.org/10.1080/01621459.2015.1044091>
- 155 de Valpine, P., Turek, D., Paciorek, C. J., Anderson-Bergman, C., Temple Lang, D., &
 156 Bodik, R. (2017). Programming with models: Writing statistical algorithms for general
 157 model structures with NIMBLE. *Journal of Computational and Graphical Statistics*, 26(2),
 158 403–413. <https://doi.org/10.1080/10618600.2016.1172487>
- 159 Finley, A. O., Datta, A., & Banerjee, S. (2022). spNNGP R package for nearest neighbor
 160 Gaussian process models. *Journal of Statistical Software*, 103(5), 1–40. [https://doi.org/](https://doi.org/10.18637/jss.v103.i05)
 161 [10.18637/jss.v103.i05](https://doi.org/10.18637/jss.v103.i05)
- 162 Finley, A. O., Datta, A., Cook, B. D., Morton, D. C., Andersen, H. E., & Banerjee, S.
 163 (2019). Efficient algorithms for Bayesian nearest neighbor Gaussian processes. *Journal*
 164 *of Computational and Graphical Statistics*, 28(2), 401–414. [https://doi.org/10.1080/](https://doi.org/10.1080/10618600.2018.1537924)
 165 [10618600.2018.1537924](https://doi.org/10.1080/10618600.2018.1537924)
- 166 Guinness, J. (2018). Permutation and grouping methods for sharpening Gaussian process
 167 approximations. *Technometrics*, 60(4), 415–429. [https://doi.org/10.1080/00401706.2018.](https://doi.org/10.1080/00401706.2018.1437476)
 168 [1437476](https://doi.org/10.1080/00401706.2018.1437476)
- 169 Ketwaroo, F. R., Muller, M. H., Saracco, J. F., & Schaub, M. (2026). An integrated
 170 population model to incorporate spatio-temporal heterogeneity in demographic rates.
 171 *bioRxiv*. <https://doi.org/10.64898/2026.03.03.709263>
- 172 Lindgren, F., Rue, H., & Lindström, J. (2011). An explicit link between Gaussian fields and
 173 Gaussian Markov random fields: The stochastic partial differential equation approach.
 174 *Journal of the Royal Statistical Society, Series B: Statistical Methodology*, 73(4), 423–498.
 175 <https://doi.org/10.1111/j.1467-9868.2011.00777.x>
- 176 Pebesma, E. (2018). Simple features for R: Standardized support for spatial vector data. *The*
 177 *R Journal*, 10(1), 439–446. <https://doi.org/10.32614/RJ-2018-009>
- 178 Risser, M. D., & Turek, D. (2020). Bayesian inference for high-dimensional nonstationary
 179 Gaussian processes. *Journal of Statistical Computation and Simulation*. <https://doi.org/>

180 [10.1080/00949655.2020.1792472](https://doi.org/10.1080/00949655.2020.1792472)

181 Rue, H., Martino, S., & Chopin, N. (2009). Approximate Bayesian inference for latent Gaussian
182 models by using integrated nested Laplace approximations. *Journal of the Royal Statistical*
183 *Society, Series B: Statistical Methodology*, 71(2), 319–392. [https://doi.org/10.1111/j.1467-](https://doi.org/10.1111/j.1467-9868.2008.00700.x)
184 [9868.2008.00700.x](https://doi.org/10.1111/j.1467-9868.2008.00700.x)

185 Saha, A., & Datta, A. (2018). BRISC: Bootstrap for rapid inference on spatial covariances.
186 *Stat*, 7(1), e184. <https://doi.org/10.1002/sta4.184>

187 Shaby, B., & Wells, M. (2011). *Exploring an adaptive Metropolis algorithm* (Technical Report
188 No. 2011-14). Department of Statistics, Duke University.

189 Vecchia, A. V. (1988). Estimation and model identification for continuous spatial processes.
190 *Journal of the Royal Statistical Society, Series B: Statistical Methodology*, 50(2), 297–312.
191 <https://doi.org/10.1111/j.2517-6161.1988.tb01729.x>

DRAFT